NKT
Tag 4: Generics, Vererbung, Binäre Suchbäume

Sven Hoops, Jonas Goudarzi

DATUM

Gliederung

1 Bäume

2 Hashing

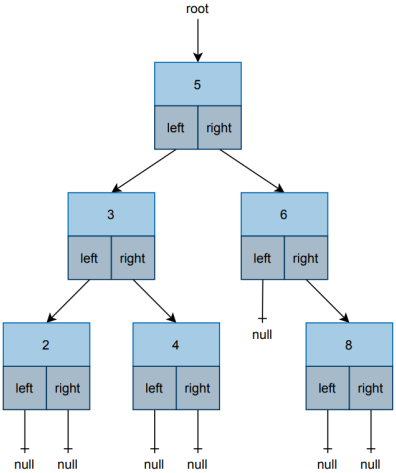
Bäume

Baum:

Bäume

Baum:

- Jedes Element (Knoten): linker und rechter Nachfolger (Kind) (= „Binärbaum“)



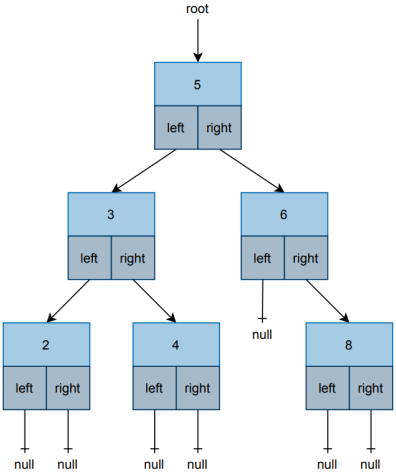
Bäume

Heinrich Heine Universität Düsseldorf

- Ordnung:
  - Alle Knoten im linken Teilbaum kleiner als Knotenwert
  - alle Knoten im rechten Teilbaum größer

  (= „binärer Suchbaum“)
- keine doppelten Werte

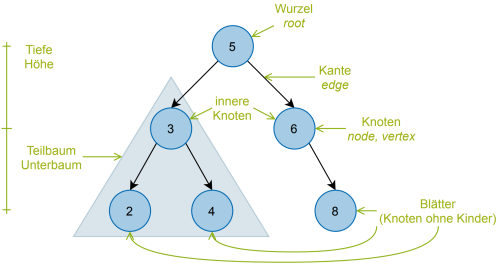
Baum:



Bäume

Baum:

- 2 ist linkes Kind von 3.
- 3 ist Elternknoten von 2 und 4.
- 3 ist die Wurzel des linken Teilbaums von 5.
- Der Baum hat die Tiefe 3.



Bäume - Umsetzung

Heinrich Heine Universität Düsseldorf

```
class BinarySearchTree {

    private class BinaryNode {
    private int element;
    private BinaryNode left, right;


    private BinaryNode(int element) {
        this.element = element;
    }
}

private BinaryNode root;
```

Bäume - Einfügen

Heinrich Heine Universität Düsseldorf

```
1  void insert(int newNumber) {
2      // Sonderfall: leerer Baum
3      if (root == null) {
4          root = new BinaryNode(newNumber);
5          return;
6      }
7
8      BinaryNode parent = null;
9      BinaryNode child = root;
10
11     while (child != null) {
12         parent = child;
13         if (newNumber == child.element) {
14             // Zahl bereits im Baum vorhanden
15             return;
16         } else if (newNumber < child.element) {
17             child = child.left;
18         } else {
19             child = child.right;
20         }
21     }
22     if (newNumber < parent.element) {
23         parent.left = new BinaryNode(newNumber);
24     } else {
25         parent.right = new BinaryNode(newNumber);
26     }
27 }
```

Bäume - Minimum finden

Heinrich Heine Universität Düsseldorf

```
int minimum() {
    if(root == null) {
        throw new java.util.NoSuchElementException();
    }

    BinaryNode current = root;
    while(current.left != null) {
        current = current.left;
    }

    return current.element;
}
```

Einschub: Exceptions

```
throw new [Exception]
```

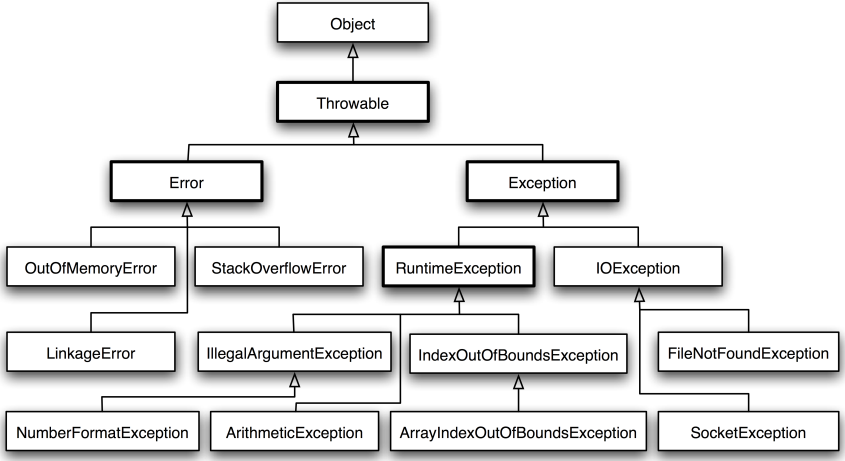
Throwing (werfen) → sorgt für das auslösen einer Exception.

```
Try{
    //Was versucht werden soll
} catch([Exceptiontyp] e) {
    // Code zur Fehlerbehandlung
}
```

- Fängt Exceptions ab
- verhindert Programmabsturz.
- Errors, können auch abgefangen werden! Ist aber fast immer eine schlechte Lösung!

Vorsicht! Alle Exceptions folgen einer Umfangreichen Vererbungshierarchie. Wenn ein zu hoher Obertyp gewählt wird, kann die tatsächliche Exception verdeckt werden oder kann nicht abgefangen werden

Einschub: Exceptions



https://www.protechtraining.com/static/bookshelf/java_fundamentals_tutorial/images/ExceptionClassHierarchy.png

Was ist Hashing?

- Hashing ist ein Verfahren, um Daten schnell zu finden.
- Es wandelt einen Schlüssel (z.B. einen String) in einen Index um.
- Dieser Index zeigt auf einen Speicherort (Bucket) in einer Hashtabelle.
- Ziel: Schneller Zugriff auf Daten (im Idealfall O(1)).

Hashfunktionen

Heinrich Heine Universität Düsseldorf

- Eine Hashfunktion $h(k)$ nimmt einen Schlüssel k und gibt einen Index zurück.
- Gute Hashfunktionen verteilen Schlüssel gleichmäßig auf die Buckets.
- Beispiele:
  - Modulo-Division: $h(k) = k \mod m$ (wobei m die Größe der Hashtabelle ist).
  - Multiplikationsmethode.
  - Universelles Hashing.

Kollisionen

- Kollision: Zwei Schlüssel werden auf denselben Bucket abgebildet.
- Kollisionen sind unvermeidbar, wenn die Anzahl der Schlüssel größer ist als die Anzahl der Buckets.
- Lösungsstrategien:
  - Verkettung (Chaining): Buckets sind Listen von Schlüsseln.
  - Offene Adressierung (Open Addressing): Suche nach einem anderen freien Bucket.
    - Lineares Sondieren (Linear Probing).
    - Quadratisches Sondieren (Quadratic Probing).
    - Doppeltes Hashing (Double Hashing).

Verkettung - Chaining

Heinrich Heine Universität Düsseldorf

- Jeder Bucket ist eine verkettete Liste.
- Bei einer Kollision wird der neue Schlüssel am Anfang oder Ende der Liste eingefügt.
- Suche: Berechne den Hashwert, durchsuche die entsprechende Liste.
- Einfügen und Löschen sind relativ einfach.

Sondierung - Linear

- Bei einer Kollision wird der nächste freie Bucket gesucht.
- Lineares Sondieren: Suche linear nach dem nächsten freien Bucket.
- Clustering-Problem: Lange Ketten von belegten Buckets können entstehen.

Sondierung Quadratisch

Heinrich Heine Universität Düsseldorf

- Bei einer Kollision wird der nächste freie Bucket gesucht.
- Quadratisch Sondieren: Suche mit x*x nach dem nächsten freien Bucket.
- Clustering-Problem: Lange Ketten von belegten Buckets können entstehen.

Anwendungen von Hashing

- Hashtabellen (Dictionaries, Maps).
- Caching.
- Kryptographie (Hashfunktionen für Passwörter, digitale Signaturen).
- Datenintegrität (Prüfsummen).
- Duplikaterkennung.

Zusammenfassung

Heinrich Heine Universität Düsseldorf

- Hashing ist ein wichtiges Verfahren für schnellen Datenzugriff.
- Hashfunktionen und Kollisionsstrategien sind entscheidend für die Effizienz.
- Hashing hat viele Anwendungen in der Informatik.

Aufgabe 1: Hashfunktion

Heinrich Heine Universität Düsseldorf

Gegeben sei eine Hashtabelle der Größe 10 und die Hashfunktion $h(k) = k \mod 10$.

1. Berechnen Sie die Hashwerte für die Schlüssel 17, 25, 38, 42, 59.
2. Zeichnen Sie die Hashtabelle nach dem Einfügen der Schlüssel unter Verwendung von Verkettung (Chaining).

Aufgabe 2: Lineares Sondieren

Heinrich Heine Universität Düsseldorf

Gegeben sei eine Hashtabelle der Größe 7 und die Hashfunktion $h(k) = k \mod 7$.

1. Fügen Sie die Schlüssel 12, 19, 26, 33, 40 in die Hashtabelle ein, indem Sie lineares Sondieren verwenden.
2. Zeichnen Sie die resultierende Hashtabelle.

Aufgabe 3: Quadratisches Sondieren

Heinrich Heine Universität Düsseldorf

Gegeben sei eine Hashtabelle der Größe 11 und die Hashfunktion $h(k) = k \mod 11$.

1. Fügen Sie die Schlüssel 14, 25, 36, 47, 58 in die Hashtabelle ein, indem Sie quadratisches Sondieren verwenden. Verwenden Sie die Sondierungsfunktion $h(k) + i^2$, wobei $i = 0, 1, 2, \ldots$.
2. Zeichnen Sie die resultierende Hashtabelle.

Aufgabe 4: Hashfunktion entwerfen

Heinrich Heine Universität Düsseldorf

Entwerfen Sie eine Hashfunktion für Strings, die aus Kleinbuchstaben bestehen. Die Hashfunktion sollte die Schlüssel gleichmäßig auf eine Hashtabelle der Größe 26 verteilen.